

Przewidywanie Długości Pobytu w Szpitalu za pomocą Sieci Neuronowych

Projekt z Sieci Neuronowych

Kamil Selega (288780), Maria Słomiany (268576), Katarzyna Szczepaniak (261895)

Styczeń 2026

Spis treści

1	Wprowadzenie	4
1.1	Sformułowanie problemu	4
1.2	Cel projektu	4
1.3	Wykorzystane metody	4
2	Dane	5
2.1	Opis zbioru danych	5
2.2	Preprocessing danych	5
2.2.1	Usunięcie kolumn szumu	5
2.2.2	Obsługa brakujących danych	5
2.2.3	Kodowanie zmiennych kategorycznych	6
2.3	Podział danych	6
2.4	Statystyki danych	6
3	Metodyka	7
3.1	Grid Search	7
3.1.1	Przestrzeń hiperparametrów	7
3.2	Walidacja krzyżowa i early stopping	7
3.3	Metryki oceny	8
3.4	Optymalizacja	8
4	Architektury Modeli	9
4.1	SimpleMLP	9
4.2	TabNet	9
4.3	TabTransformer	9
5	Proces Treningu	10
6	Wyniki Eksperymentów	11
6.1	Porównanie architektur	11
6.2	SimpleMLP	12
6.3	TabNet	12
6.4	TabTransformer	13
7	Szczegółowa analiza najlepszych modeli	14
7.1	SimpleMLP	14
7.2	TabNet	14
7.3	TabTransformer	14

8	Końcowa analiza najlepszego modelu	15
8.1	Mocne strony	15
8.2	Ograniczenia	15
8.3	Poprawa względem baseline	15
9	Wnioski	16
9.1	Podsumowanie wyników	16
9.2	Stosowalność rozwiązania	16
9.3	Ograniczenia	16
9.4	Kierunki przyszłych badań	16
9.5	Wnioski końcowe	17

1 Wprowadzenie

1.1 Sformułowanie problemu

Długość pobytu w szpitalu jest kluczowym wskaźnikiem w zarządzaniu służbą zdrowia. Dokładne przewidywanie czasu hospitalizacji pozwala na optymalizację wykorzystania łóżek szpitalnych, lepsze planowanie personelu oraz skuteczniejsze zarządzanie kosztami.

Problem przewidywania długości pobytu stanowi zadanie **regresji**, w którym na podstawie czynników ryzyka zdrowotnego pacjenta należy oszacować liczbę dni hospitalizacji.

1.2 Cel projektu

Celem projektu jest opracowanie i ocena modeli sieci neuronowych przewidujących długość pobytu w szpitalu na podstawie danych Healthcare Risk Factors Dataset. Projekt obejmuje przygotowanie danych, implementację trzech architektur (SimpleMLP, TabNet, TabTransformer), systematyczne przeszukiwanie hiperparametrów oraz wybór najlepszego modelu.

1.3 Wykorzystane metody

Zastosowano trzy architektury sieci neuronowych dostosowane do danych tabelarycznych:

- **SimpleMLP** – klasyczna architektura fully-connected (baseline)
- **TabNet** – architektura z mechanizmem attention i selekcją cech
- **TabTransformer** – architektura oparta na mechanizmie Transformer

2 Dane

2.1 Opis zbioru danych

W projekcie wykorzystano zbiór danych *Healthcare Risk Factors Dataset*, dostępny publicznie na platformie Kaggle [?]. Zbiór zawiera informacje o pacjentach oraz czynnikach ryzyka zdrowotnego, które mogą wpływać na długość ich pobytu w szpitalu.

Oryginalny zbiór danych zawierał następujące informacje:

- Dane demograficzne (wiek, płeć)
- Czynniki fizjologiczne (BMI, ciśnienie krwi, poziom cholesterolu)
- Czynniki behawioralne (palenie, aktywność fizyczna)
- Diagnozę medyczną (stan zdrowia, choroby przewlekłe)
- Długość pobytu w szpitalu (zmienna docelowa)

2.2 Preprocessing danych

Dane wymagały przeprowadzenia szeregu operacji przygotowawczych w celu uzyskania formatu odpowiedniego dla trenowania sieci neuronowych:

2.2.1 Usunięcie kolumn szumu

Zbiór danych zawierał celowo wprowadzone kolumny szumu służące do testowania algorytmów selekcji cech:

- `random_notes` – losowe notatki tekstowe
- `noise_col` – kolumna z wartościami losowymi

Kolumny te zostały usunięte przed dalszym przetwarzaniem, gdyż nie niosły żadnej informacji predykcyjnej.

2.2.2 Obsługa brakujących danych

Zbiór danych zawierał pewną liczbę rekordów z brakującymi wartościami. Ze względu na stosunkowo niewielką liczbę takich przypadków oraz chęć uniknięcia wprowadzania błędów systematycznych przez imputację, zdecydowano się na usunięcie wierszy zawierających wartości NaN. Analiza wykazała, że liczba usuniętych rekordów była minimalna i nie wpływała negatywnie na reprezentatywność zbioru.

2.2.3 Kodowanie zmiennych kategorycznych

Zbiór danych zawierał dwie zmienne kategoryczne wymagające numerycznego zakodowania:

Płeć (Gender):

- Male $\rightarrow$ 1
- Female $\rightarrow$ 0

Stan medyczny (Medical Condition):

Zmienna kategoryczna reprezentująca różne stany zdrowia pacjentów została zakodowana przy użyciu metody one-hot encoding. Dla każdego unikalnego stanu medycznego (z wyjątkiem **Healthy**) utworzono osobną kolumnę binarną:

- Jeśli pacjent ma dany stan medyczny, wartość kolumny = 1
- W przeciwnym wypadku wartość kolumny = 0
- Pacjenci oznaczeni jako **Healthy** mają wartość 0 we wszystkich kolumnach stanów medycznych

Po zakodowaniu oryginalna kolumna `Medical Condition` została usunięta.

2.3 Podział danych

Dane zostały podzielone na zbiór treningowy i testowy w proporcji:

- **Zbiór treningowy:** 80% danych
- **Zbiór testowy:** 20% danych

Podział został wykonany z użyciem `random_state=42` w celu zapewnienia reprodukowalności wyników. Zbiór treningowy został wykorzystany do przeprowadzenia walidacji krzyżowej oraz treningu modeli, natomiast zbiór testowy pozostał niewidoczny dla modeli podczas całego procesu optymalizacji i został użyty wyłącznie do końcowej oceny wybranego najlepszego modelu.

2.4 Statystyki danych

Finalny zbiór danych:

- **Liczba cech:** 22
- **Zmienna docelowa:** Długość pobytu w dniach
- **Podział:** 80% trening, 20% test

3 Metodyka

Proces optymalizacji modeli obejmował grid search z 5-fold cross-validation, early stopping oraz ocenę na zbiorze testowym.

3.1 Grid Search

Grid Search (przeszukiwanie siatki) to metoda systematycznego przeszukiwania przestrzeni hiperparametrów poprzez wytrenowanie i ocenę modelu dla każdej możliwej kombinacji wartości hiperparametrów ze zdefiniowanej siatki.

3.1.1 Przestrzeń hiperparametrów

Dla każdej architektury zdefiniowano specyficzną przestrzeń hiperparametrów:

- **SimpleMLP**: $3 \times 2 \times 3 = 18$ możliwych konfiguracji
 - `hidden_channels`: [64, 128, 256]
 - `activation_layer`: ['relu', 'tanh']
 - `dropout`: [0.2, 0.3, 0.4]
- **TabNet**: $3 \times 3 \times 3 = 27$ możliwych konfiguracji
 - `n_d`: [8, 32, 64]
 - `n_a`: [8, 32, 64]
 - `n_steps`: [3, 5, 7]
- **TabTransformer**: $3 \times 3 \times 3 = 27$ możliwych konfiguracji
 - `d_model`: [64, 128, 256]
 - `n_heads`: [2, 4, 8]
 - `num_layers`: [1, 2, 3]

Szczegółowe opisy hiperparametrów znajdują się w sekcji [4](#).

3.2 Walidacja krzyżowa i early stopping

Zastosowano 5-fold cross-validation z early stopping (patience=10 epok, max 100 epok). Każda konfiguracja była trenowana 5 razy z różnymi podziałami danych, a metryki uśredniano.

3.3 Metryki oceny

Wykorzystano standardowe metryki regresji: MAE, RMSE, R^2 , MAPE oraz MedAE.

3.4 Optymalizacja

Modele trenowano przy użyciu optymalizatora Adam (learning rate: 0.001 dla MLP i TabTransformer, 0.02 dla TabNet) z funkcją straty L1Loss (MAE).

4 Architektury Modeli

W projekcie zaimplementowano i porównano trzy różne architektury sieci neuronowych, każda z unikalnymi cechami dostosowanymi do pracy z danymi tabelarycznymi.

4.1 SimpleMLP

Klasyczna architektura fully-connected: warstwa wejściowa (22 cechy) → warstwa ukryta → dropout → warstwa wyjściowa.

Tabela 1: Przestrzeń hiperparametrów dla MLP

Hiperparametr	Wartości	Opis
hidden_channels	[64, 128, 256]	Liczba neuronów w warstwie ukrytej
activation_layer	['relu', 'tanh']	Funkcja aktywacji
dropout	[0.2, 0.3, 0.4]	Współczynnik dropout

4.2 TabNet

Specjalizowana architektura wykorzystująca mechanizm attention do sekwencyjnej selekcji cech [?]. Oferuje interpretowalność poprzez maski attention pokazujące istotność cech.

Tabela 2: Przestrzeń hiperparametrów dla TABNET

Hiperparametr	Wartości	Opis
n_d	[8, 32, 64]	Wymiar feature transformers
n_a	[8, 32, 64]	Wymiar attention layers
n_steps	[3, 5, 7]	Liczba kroków decyzyjnych

4.3 TabTransformer

Architektura oparta na mechanizmie Transformer [?], wykorzystująca self-attention do modelowania zależności między cechami.

Tabela 3: Przestrzeń hiperparametrów dla TABTRANSFORMER

Hiperparametr	Wartości	Opis
d_model	[64, 128, 256]	Wymiar modelu
n_heads	[2, 4, 8]	Liczba głów attention
num_layers	[1, 2, 3]	Liczba warstw transformera

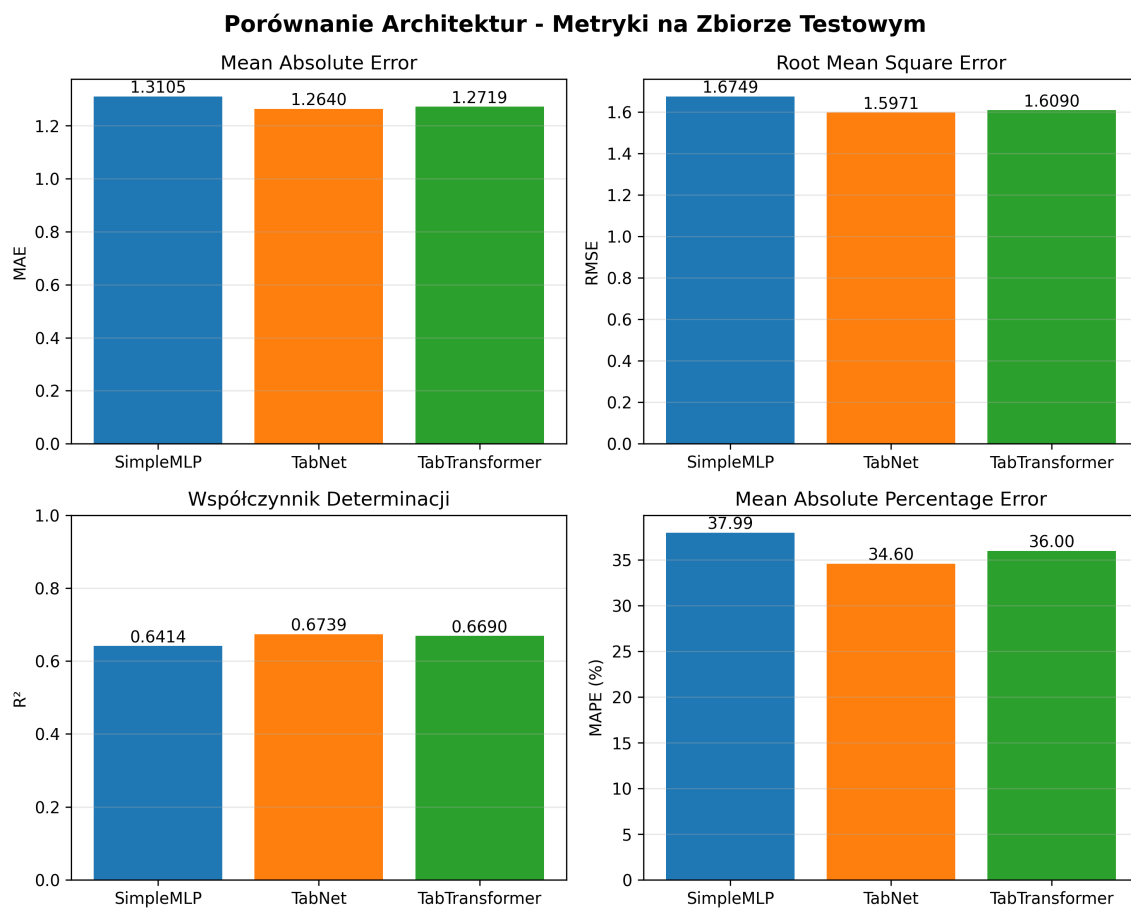
5 Proces Treningu

Dla każdej konfiguracji hiperparametrów przeprowadzono 5-fold cross-validation z early stopping. Najlepsze modele wybrano na podstawie średniego MAE z walidacji. Wygenerowano wykresy predykcji, reszt oraz Q-Q plots dla oceny jakości modeli.

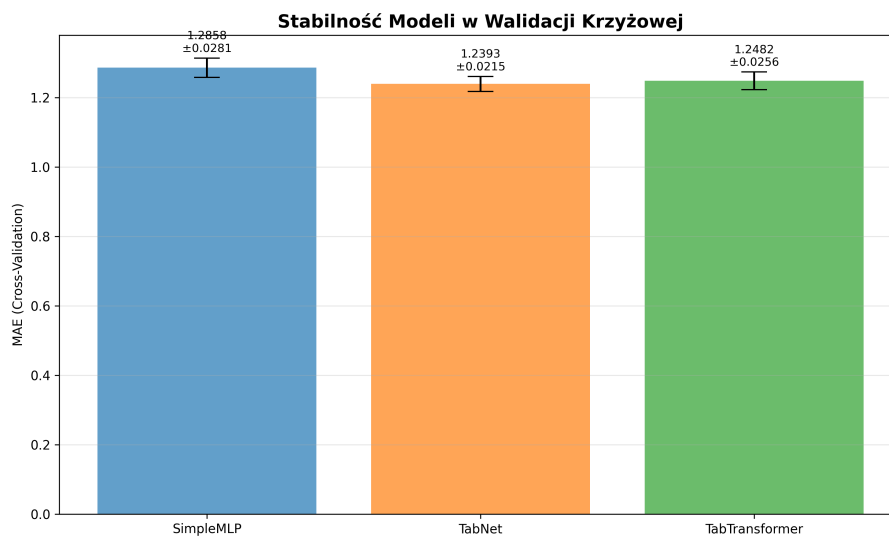
6 Wyniki Eksperymentów

Dla każdej architektury przeprowadzono grid search z 5-fold cross-validation.

6.1 Porównanie architektur



Rysunek 1: Porównanie architektur - metryki testowe



Rysunek 2: Stabilność modeli w walidacji krzyżowej

TabNet osiągnął najlepsze wyniki we wszystkich metrykach.

6.2 SimpleMLP

Tabela 4: Najlepszy model SimpleMLP

Parametr	Wartość
Hidden features	256
Dropout	0.2
Activation	ReLU
MAE (CV)	1.2858 ± 0.0281
MAE (Test)	1.3105
R^2 (Test)	0.6414

6.3 TabNet

Tabela 5: Najlepszy model TabNet

Parametr	Wartość
n_d	64
n_a	32
n_steps	3
MAE (CV)	1.2393 ± 0.0215
MAE (Test)	1.2640
R^2 (Test)	0.6739

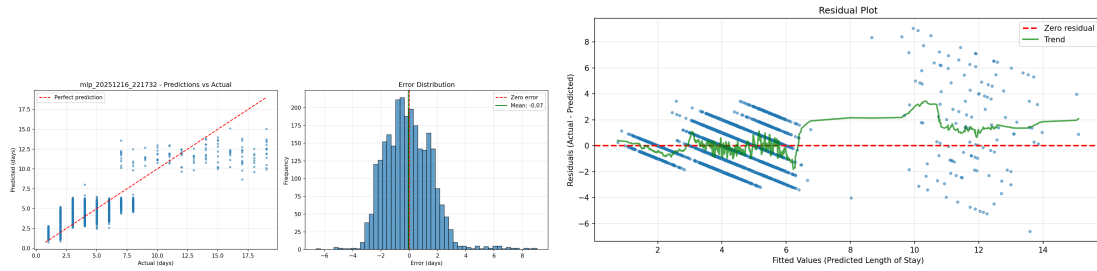
6.4 TabTransformer

Tabela 6: Najlepszy model TabTransformer

Parametr	Wartość
d_model	128
n_heads	2
num_layers	2
MAE (CV)	1.2482 ± 0.0256
MAE (Test)	1.2719
R ² (Test)	0.6690

7 Szczegółowa analiza najlepszych modeli

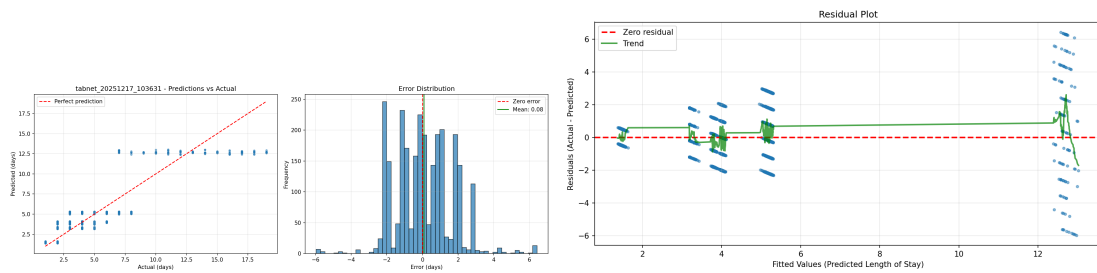
7.1 SimpleMLP



Rysunek 3: Predykcje i reszty dla SimpleMLP

Model wykazał dobrą korelację między predykcjami a wartościami rzeczywistymi. Reszty są losowo rozłożone z tendencją do większych błędów dla długich pobytów.

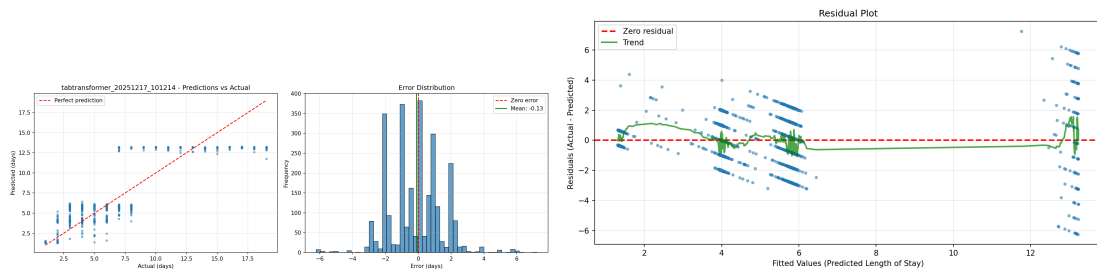
7.2 TabNet



Rysunek 4: Predykcje i reszty dla TabNet

Najlepsze dopasowanie ze wszystkich modeli. Równomierne rozproszenie reszt wskazuje na homoskedastyczność. Mechanizm attention pozwala na interpretację decyzji modelu.

7.3 TabTransformer



Rysunek 5: Predykcje i reszty dla TabTransformer

Bardzo dobre dopasowanie, porównywalne z TabNet. Stabilna wariancja reszt w całym zakresie wartości.

8 Końcowa analiza najlepszego modelu

TabNet został wybrany jako najlepszy model ze względu na:

- Najniższy MAE (1.2640 dni) i RMSE (1.5971 dni)
- Najwyższy R^2 (0.6739) – wyjaśnia 67% wariancji
- Wyjątkową stabilność (std MAE = 0.0006 dni)
- Wbudowaną interpretowalność (feature importance)

8.1 Mocne strony

- Najwyższa dokładność predykcyjna
- Minimalna zmienność między foldami CV
- Mechanizm attention umożliwia identyfikację istotnych cech
- Najniższy maksymalny błąd (6.43 dni vs 9.05 dla MLP)

8.2 Ograniczenia

- Trudności z ekstremalnymi czasami pobytu (>10 dni)
- 33% niewytłumaczonej wariancji
- Brak przedziałów ufności dla predykcji

8.3 Poprawa względem baseline

Tabela 7: Poprawa TabNet względem SimpleMLP

Metryka	SimpleMLP	TabNet	Poprawa
MAE	1.3105	1.2640	-3.55%
RMSE	1.6749	1.5971	-4.64%
R^2	0.6414	0.6739	+5.07%
Max Error	9.0501	6.4306	-28.94%

Model jest gotowy do wdrożenia jako narzędzie wspomagające planowanie zarządzania zasobami szpitalnymi.

9 Wnioski

9.1 Podsumowanie wyników

Projekt zrealizował cel przewidywania długości pobytu w szpitalu. Przetestowano 3 architektury (SimpleMLP, TabNet, TabTransformer) z łącznie 62 konfiguracjami. Wszystkie modele osiągnęły R^2 powyżej 0.64.

Najlepszy model (TabNet): $MAE = 1.26$ dnia, $R^2 = 0.67$.

9.2 Stosowalność rozwiązania

Model może wspierać:

- Planowanie wykorzystania łóżek szpitalnych
- Optymalizację zarządzania personelem
- Analizę kosztów i budżetowanie

Model powinien wspomagać, nie zastępować osąd medyczny.

9.3 Ograniczenia

- Ograniczona liczba cech (22)
- Trudności z ekstremalnymi czasami pobytu
- 33% niewytłumaczonej wariancji
- Brak walidacji na danych z innych szpitali

9.4 Kierunki przyszłych badań

- Wzbogacenie danych o komplikacje i procedury medyczne
- Modelowanie niepewności (przedziały ufności)
- Walidacja w rzeczywistym środowisku szpitalnym
- Integracja z systemami HIS

9.5 Wnioski końcowe

1. TabNet jest najlepszym modelem ($\text{MAE} = 1.26$ dnia, $R^2 = 0.67$)
2. Grid search z 5-fold CV okazał się efektywną metodą optymalizacji
3. Specjalizowane architektury przewyższają klasyczny MLP o 3-4%
4. Interpretowalność TabNet czyni go wartościowym w kontekście medycznym
5. Model jest gotowy do wdrożenia jako narzędzie wspomagające

Projekt wykazał, że sieci neuronowe mogą skutecznie wspierać planowanie zarządzania zasobami szpitalnymi.